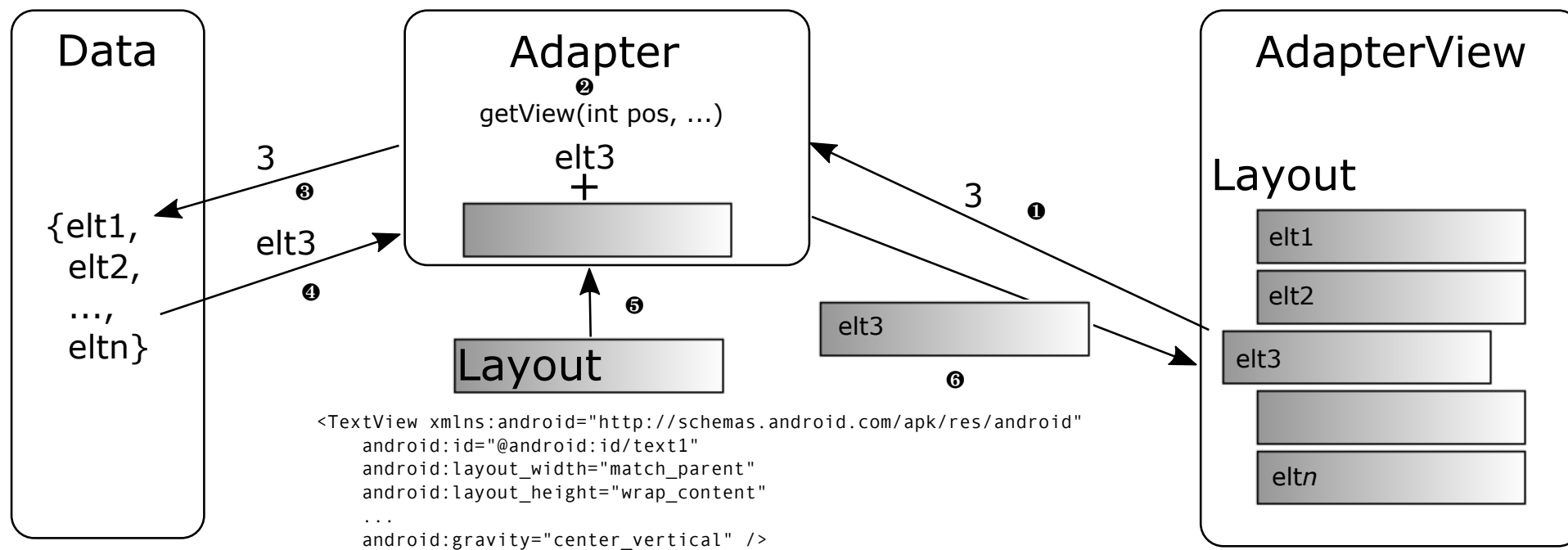


ViewGroup

- ViewGroup : composant pour afficher plusieurs objets similaires
- constituants :
 - 1 données : souvent liste ou tableau d'objets (Personne, Note, ...)
 - 2 layouts :
 - ★ disposition DES éléments (ex : liste ou grille)
 - ★ xml de la présentation d'UN élément
 - 3 adapter : pont entre données et AdapterView, fabrique la vue représentant un élément des données

Adapter



Adapter

- prototype de la méthode essentielle,
`View getView (int position, View convertView, ViewGroup parent`
- fournit une `View` (i.e. un composant graphique) représentant l'élément `position` d'une structure de données
- cette `View` provient (le + souvent) d'un layout xml

ArrayAdapter

- implantation la plus simple d'un `Adapter`
- une `List<T>` pour les données (alimentée par un tableau ou une liste)
- un `TextView` pour l'affichage de la donnée
- solutions pour l'utiliser :
 - 1 layout = UN seul `TextView.toString()` de la donnée comme texte affiché
 - 2 layout personnalisé : sous classer `ArrayAdapter` pour redéfinir `View getView (int position, View convert, ViewGroup parent`

ArrayAdapter : tableau de chaîne

- AdapterView utilisé : une ListView dont le layout est un LinearLayout

```
<ListView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/ListView1"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >
</ListView>
```

- données (modèle) : tableau de chaîne de caractère
- layout de la vue : android.R.layout.simple_list_item_1 (layout d'un item de la liste)

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(dev.mobile.cmlistview.R.layout.activity_listview)

    val tm = arrayOf("Leibniz", "Descartes", "Euler")
    val aa = ArrayAdapter<String>(this, android.R.layout.simple_list_item_1, tm)
    val lv = findViewById<ListView>(R.id.mathematicians)
    lv.adapter = aa
}
```

android.R.layout.

- il existe des layouts tout fait
- *platforms/android-NN/data/res/layout/simple_list_item_1.xml*

```
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@android:id/text1"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textAppearance="?android:attr/textAppearanceListItemSmall"
    android:gravity="center_vertical"
    android:paddingStart="?android:attr/listPreferredItemPaddingStart"
    android:paddingEnd="?android:attr/listPreferredItemPaddingEnd"
    android:minHeight="?android:attr/listPreferredItemHeightSmall" />
```

Adapter : liste d'objets

- donnée (modèle) : liste d'objet (ici une classe `Personne`)
- layout de la vue : `android.R.layout.simple_list_item_1`

```
val lp = mutableListOf(  
    Personne("Descartes", 1596, 3, 31),  
    Personne("Euler", 1707, 4, 15)  
)
```

```
val aa = ArrayAdapter(this, android.R.layout.simple_list_item_1, lp)
```

```
val lv = findViewById<ListView>(R.id.mathematicians)  
lv.adapter = aa  
aa.add(Personne("Leibniz", 1646, 7, 1)) // OK  
lp.add(Personne("Gauss", 1777, 4, 30)) // OK
```

LayoutInflater

LayoutInflater :

- objet pour instancier une hiérarchie d'objets `View` correspondant à un layout
- analyse et conversion du fichier xml de layout
- méthode `inflate` renvoie une instance de `View`

```
inflater.inflate(R.layout.item, parent, false)
```

AdapterView : layout personnalisé

- définition d'un fichier `ligne_layout.xml`
- ici **DEUX** `TextView` : "`@+id/nom`" et "`@+id/naissance`"

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/LinearLayout1"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal" >
    <ImageView
        android:id="@+id/imageView1"
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:src="@android:drawable/ic_delete" />
    <TextView
        android:id="@+id/nom"
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:textAppearance="?android:attr/textAppearanceLarge" />
    <TextView
        android:id="@+id/naissance"
        android:layout_width="wrap_content"
        android:layout_height="match_parent" />
</LinearLayout>
```

AdapterView : layout personnalisé

- définition de la classe Adapter à partir de ArrayAdapter

```
class PersonneAdapterC(context: Context, items: MutableList<Personne>) :  
    ArrayAdapter<Personne>(context, 0, items) {  
    override fun getView(position: Int, convertView: View?, parent: ViewGrou  
        val row: View  
  
        val personne = getItem(position)  
  
        val inflater = LayoutInflater.from(context)  
  
        row = inflater.inflate(R.layout.item, parent, false)  
  
        if (personne != null) {  
            row.findViewById<TextView>(R.id.nom).text = personne.nom  
            row.findViewById<TextView>(R.id.naissance).text = personne.getNais  
        }  
        return row  
    }  
}
```

AdapterView : layout personnalisé

- association entre AdapterView et Adapter

```
val lp = mutableListOf<Personne>(
    Personne("Descartes", 1596, 3, 31),
    Personne("Euler", 1707, 4, 15),
    Personne("Leibniz", 1646, 7, 1))
val lv = findViewById<ListView>(R.id.mathematicians)
pa = PersonneAdapterC(this, lp)
lv.adapter = pa
```

- recréer systématiquement des vues pour les items coûteux
- `getView` reçoit un paramètre permettant de recycler des `View`
- on modifie (juste) le contenu

ViewHolder – get/setTag

- opération `findViewById` coûteuse
- utilisation des méthodes `getTag/setTag` pour associer un objet à une `View`
- on définit cet objet pour référencer les `View` enfants
- on utilise une classe interne (statique) :

```
internal class ViewHolder {  
    lateinit var nom: TextView  
    lateinit var naissance: TextView  
}
```

- on peut également référencer l'adaptateur

ViewHolder

```
var row = convertView
val viewHolder: ViewHolder

if (row == null) {
    row = inflater.inflate(R.layout.item, parent, false)
    viewHolder = ViewHolder()
    viewHolder.nom = row.findViewById<TextView>(R.id.nom)
    viewHolder.naissance = row.findViewById<TextView>(R.id.naissance)
    row.tag = viewHolder
} else {
    viewHolder = row.tag as ViewHolder
}

viewHolder.nom.text = getItem(position)?.nom
viewHolder.naissance.text = getItem(position)?.getNaiss()

return row as View
```

RecyclerView

- Un intégré de ce qui précède
- quelques points de difficulté : en particulier la gestion des événements

Réseau

- sources diverses :

- 1 `java.net` : `Socket`, `URL` ($\rightarrow$ IO) : laborieux pour HTTP
- 2 `org.apache` : `HttpRequest`, `HttpResponse` : **dépréciés**
- 3 `android.net` : `URI`, `java.net` : `HttpURLConnection`
- 4 **bibliothèques tierces :**
 - ★ *Volley*
(<https://developer.android.com/training/volley/index.html>)
 - ★ *Ion*
(<https://github.com/koush/ion>)
 - ★ *Android Asynchronous Http Client*
(<http://loopj.com/android-async-http/>)
 - ★ *Retrofit/okHttp*
(<https://square.github.io/retrofit>)
(<https://square.github.io/okhttp>)
 - ★ ...

- nécessite des permissions :

```
<uses-permission android:name="android.permission.INTERNET" />  
<uses-permission android:name="android.permission.ACCESS_NETWORK_S
```

Volley : GET

```
val url = "http://site.com/unePage.html"
val stringRequest = StringRequest(
    Request.Method.GET,
    url,
    Response.Listener<String> { response ->
        tv_visu.text = "Réponse : \n ${response}"
    },
    Response.ErrorListener { error ->
        tv_visu.text = "Problème : $error"
    })

queue.add(stringRequest)
```

Volley : POST

```
val url = "http://site.com/formPost.php"
val stringRequest = object : StringRequest(
    Method.POST,
    url,
    { response -> tv_visu.text = "Réponse : \n ${response}" },
    { error -> tv_visu.text = "Problème : $error" }
) {
    override fun getParams(): Map<String, String>? {
        val params: MutableMap<String, String> = HashMap()
        params["nom"] = "Lanoix"
        params["prenom"] = "Arnaud"
        return params
    }

    override fun getHeaders(): Map<String, String> {
        val params: MutableMap<String, String> = HashMap()
        params["Accept"] = "text/html"
        params["Accept-Language"] = "fr"
        return params
    }
}
queue.add(stringRequest)
```

Json

- données packagées en deux structures :

- 1 couples clé/valeur
- 2 listes ordonnées de valeur

- exemple :

```
{  
  "myNumber": 123.456,  
  "myString": "abcdef",  
  "myBool": true,  
  "myArray": [ "a", "b", "c", "d" ],  
  "myObject": { "id": 7 },  
  "myNull": null  
}
```

<https://www.json.org/json-fr.html>

Json

org.json

JSONObject
+get(String): Object +getJSONObject(String): JSONObject +getJSONArray(String): JSONArray +getInt(String): int +getString(String): String ...

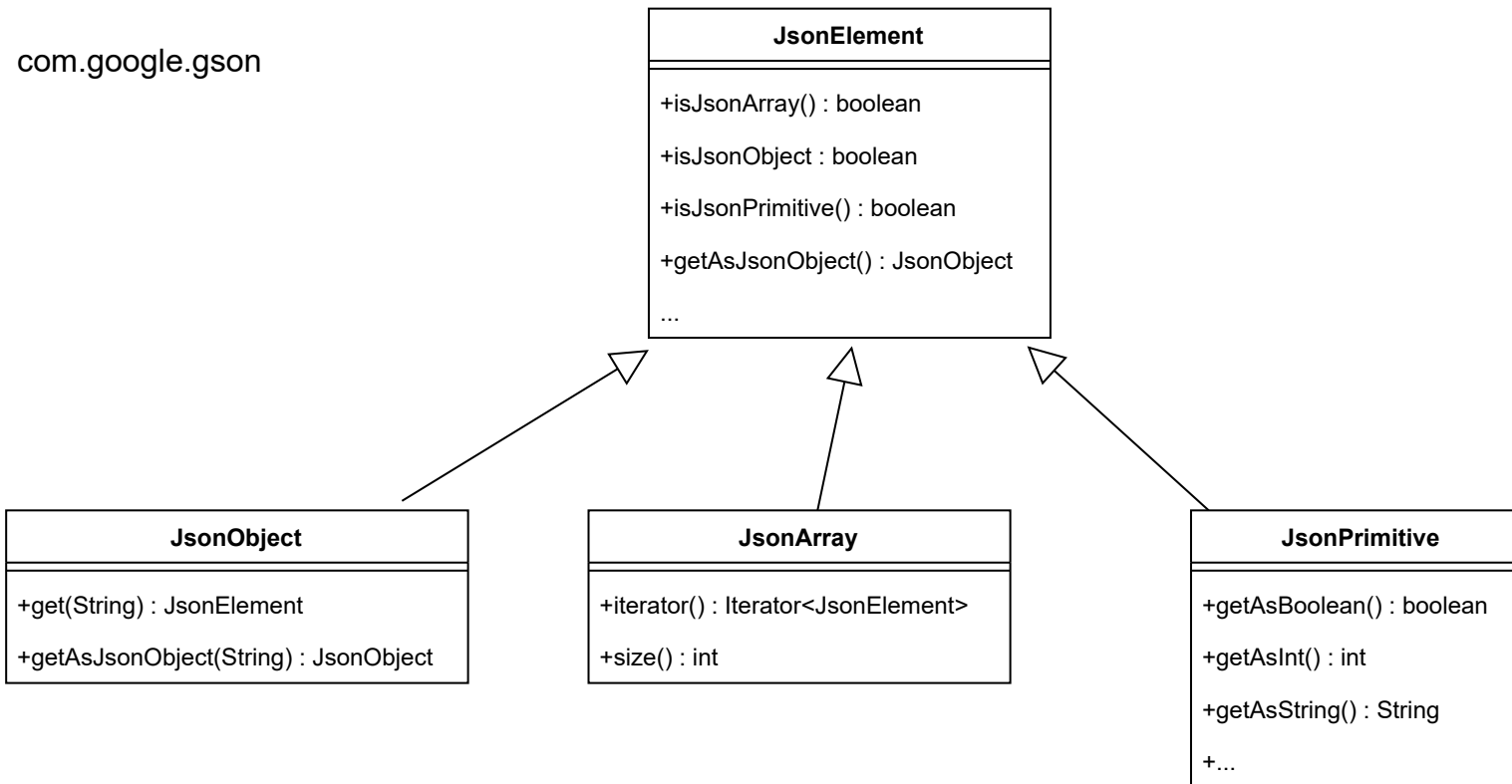
JSONArray
+get(int): Object +getJSONObject(int): JSONObject +getJSONArray(int): JSONArray +getInt(int): int +getString(int): String +length():int +iterator():Iterator<Object> ...

Json : exemple

`https://github.com/stleary/JSON-java`

`// TODO`

Json : API concurentes



Json String $\longleftrightarrow$ Pojo (*Plain old Java object*) / kotlin *data class*

- gson (<https://github.com/google/gson>),
- moshi (<https://github.com/square/moshi>),
- jackson (<https://github.com/FasterXML/jackson>),
- Kotlinx-serialisation-json
(<https://github.com/Kotlin/kotlinx.serialization>),
- ...

Persistence

Plusieurs solutions possibles :

- ① fichier : privé à l'application ou partagé
- ② map : association clés/valeurs improprement appelée *Shared preferences*
- ③ SGBD sqlite : bases privées à l'application. API très bas niveau ou via un ORM : Room, OrmLite, ...
- ④ autres bd : Realm

Fichier

- il est possible d'utiliser l'API java `File`
- Création

```
val file = File(context.filesDir, filename)
```

- puis on utilise le stream approprié :

```
val fos = FileOutputStream(file)
```

- ou directement `openFileOutput()` de `Context`
- exemple :

```
val filename = "myfile"
val fileContents = "Hello world!"
context.openFileOutput(filename, Context.MODE_PRIVATE).use {
    it.write(fileContents.toByteArray())
}
```

<https://developer.android.com/training/data-storage/app-specific>

Shared preferences

- stockage d'une map clé-valeur
- limité aux types simples et chaîne
- écriture :

```
SharedPreferences sharedPref =  
    context.getSharedPreferences("DEMO", Context.MODE_PRIVATE);  
SharedPreferences.Editor editor = sharedPref.edit();  
editor.putInt("CLICK", ++numClick);  
editor.commit();
```

- lecture :

```
SharedPreferences sharedPref =  
context.getSharedPreferences("DEMO", Context.MODE_PRIVATE);  
int val = sharedPref.getInt("CLICK", -1);
```

- fichier XML

(/data/data/dev.mobile.cmsp/shared_prefs/DEMO.xml)

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>  
<map>  
    <int name="CLICK" value="2" />  
</map>
```

<https://developer.android.com/training/data-storage/shared-preferences>

Room

- Entity : définition du schéma de la base
- DAO (*Database Access Object*) : définition des opérations sur la base
- Database

Entity

- classe = table
- instance = ligne de donnée
- attribut = nom de colonne
- utilisation d'annotations
- getter/setter pour accès

```
@Entity
data class Task (
    @PrimaryKey(autoGenerate = true)
    private val id,
    private val title: String,
    private val priority,
    private val completed ,
)
```

<https://developer.android.com/training/data-storage/room/defining-data>

DAO

- classe d'accès aux données
- annotation :
 - @Delete : l'implantation de la méthode supprime ses paramètres de la base
 - @Insert : l'implantation de la méthode insère ses paramètres dans la base
 - @Query : requête ordinaire

```
@Dao
interface TaskDAO {
    @Insert
    fun insert(tache: Task)

    @Delete
    fun delete(tache: Task)

    @Update
    fun update(tache: Task)

    @get:Query("select * from Task order by priority desc")
    val allTask: List<Task>
}
```

<https://developer.android.com/training/data-storage/room/accessing-data>

Database

```
@Database(entities = arrayOf(Task::class), version = 1, exportSchema = false)
abstract class TaskRoomDatabase : RoomDatabase() {

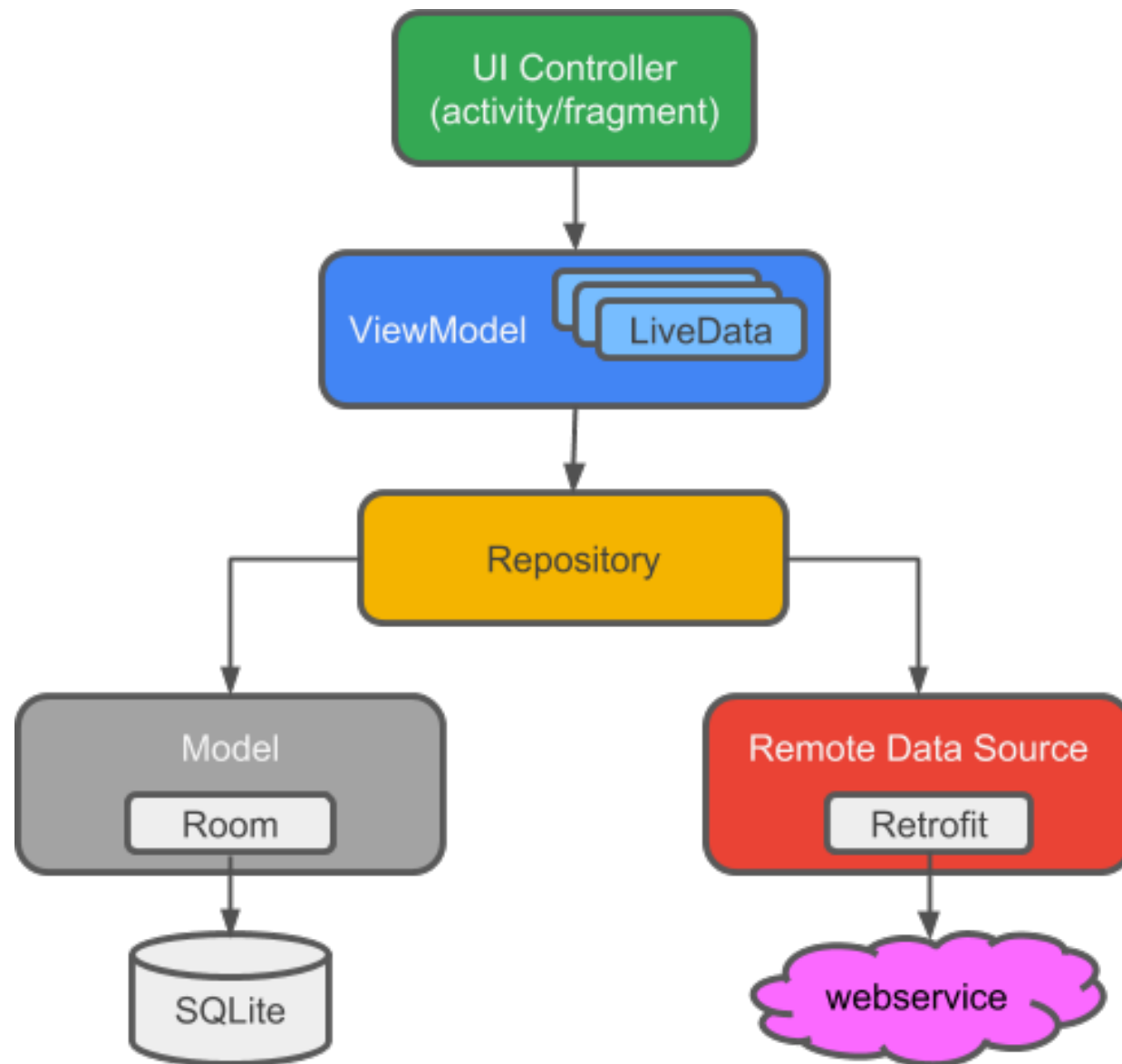
    abstract fun taskDao(): TaskDao

    companion object {

        @Volatile
        private var INSTANCE: TaskRoomDatabase? = null

        fun getDatabase(context: Context): TaskRoomDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    TaskRoomDatabase::class.java,
                    "task_database"
                ).build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

Architecture



Conclusion ?

- ce que nous n'avons pas vu d'important :
 - Architecture conseillée
 - `BroadcastReceiver`, `ContentProvider`, `Service`
 - capteurs : accéléromètre, gyroscope, électromagnétisme, orientation, ...
 - gestes et multitouch
 - Multimédia et graphisme

Références

- <https://developer.android.com/guide/index.html>